

Performance

Building agents by building their harness

Lesson 9 of 10 · Production

Where we left off

The agent survives failures.

But it's slow and it's expensive.

Why it's slow

Each step is a model call.

A task is many steps.

You wait on every one.

Why it's expensive

Lesson 3: each request carries **everything so far**.

Step 10 resends steps 1 to 9.

Performance

Make it faster and cheaper without changing what it does.

Where does it live?

Performance is spread across **several primitives**.

Three places in quark.py

- **context**: prompt caching, small requests
- **model interface**: streaming, a fixed smaller model for summaries
- **output**: text shown as it arrives, tools run in parallel

And one more place

In `performance.py`, choosing a model **per task** is **control flow**.

Routing, from Lesson 3.

We'll get there.

The worked example

`production/09-performance/quark.py`

Lesson 8's `quark.py` plus a performance layer.

159 lines.

Start with context

The expensive part

Each request begins with the same things:
instructions, then the conversation so far.

The model **reads all of it again.**

Prompt caching

The API remembers the **start** of a request.

Send the same start again: it costs far less.

How a match works

The cache matches a **prefix**.

Same beginning, same tokens, same order.

One changed token breaks the match from there on.

Why Lesson 4 had no time

The system prompt says today's **date**, not the time.

A timestamp would change every call.

The prefix would never match.

Marking a breakpoint

`cache_control` marks where the cached part **ends**.

Lesson 4 put one on the system prompt.

The conversation grows

The system prompt is cached.

The conversation is not.

Every step resends all of it at full price.

A second breakpoint

```
cached(working_memory)
```

Puts `cache_control` on the **last block of the last message**.

Why the last message

Next step, the prefix is longer by one turn.

Everything up to here is **already cached**.

Only the new part is paid in full.

The numbers: write and read

Writing to the cache costs about **1.25x**.

Reading from it costs about **0.1x**.

The catches

- the cache lasts about **5 minutes**
- there is a **minimum size** to cache
- the cache is **per model**

Seeing it work

Lesson 5's trace has `cache_read` and `cache_write` .

The first call writes the prompt to the cache, unless a run minutes before already did.

Each step after

In the README's run: **read** 5,182, then 5,311, then 5,466.

Written each time: only what's new, 129, 155 and 119 tokens.

Observability pays off

You can't tune what you can't see.

The trace made this a **measurement**.

Smaller requests

Cheaper still: **send less.**

Trimming tool results

```
MAX_RESULT = 20_000
```

A long result is cut before it enters working memory.

Head and tail

Keep the **start** and the **end**.

Cut the middle.

Both are where the useful part usually is.

Say what was cut

```
[... 5327 characters cut ...]
```

The model knows something is missing.

It can ask for more.

Seen in the README

A 25,327-character result.

Trimmed to 20,000.

Lesson 4 had this

```
trim() in context.py .
```

Now it's in the working agent.

Now the model interface

Waiting for the whole reply

Without streaming: nothing until it's done.

Then it all appears.

Streaming

```
client.messages.stream
```

Text arrives **as it's generated**.

Lesson 1 already streamed

For a different reason: long responses time out.

Nothing was printed.

Now we print

The text goes to the terminal **live**.

Receiving the stream is the model interface. Showing it to you is output.

Same total time. Feels much faster.

Latency you feel

The time to the **first word** is what you notice.

Not the total.

ask()

```
ask(models=MODELS, live=False, **request)
```

One function for every model call.

live

`live=True` : print the text as it arrives, for the person.

`live=False` : collect quietly.

Other requests don't need live

A summary is not for a person to watch.

So `compact()` calls with `live=False`.

A model per request

```
FAST = ["claude-haiku-4-5"]
```

Compaction uses FAST

Summarizing doesn't need the strongest model.

```
compact() passes mode ls=FAST .
```

Faster. Cheaper.

A fixed choice

In `quark.py`, this is one fixed choice for one job.

Nothing is **deciding**.

A caveat

The small model may have a **smaller window**.

If the summary request is too long, Lesson 4's `drop` path takes over.

A caveat on our testing

The README only tested compaction on the small model with a direct call.

Not a full-window run.

It says so.

Now output

Tools run one at a time

The model asks for three commands.

We run them in order.

Three waits, added up.

They may not depend on each other

```
ls , wc , git status .
```

No need to wait for one to start the next.

Lesson 2 hinted at this

`input_output.py` used `asyncio.gather` .

Three 2-second tasks took 2 seconds, not 6.

First: tell the model

The system prompt used to say **one command per response**.

Now: commands that don't depend on each other can go in the same response.
They **run at the same time**.

Why that matters

The old line told the model *not* to ask for several at once.

A harness can only run in parallel what the model asks for in parallel.

Changing context can change performance.

Three passes

Pass 1: in order: cut off, refused, or approved.

Pass 2: the approved commands, all at once.

Pass 3: the results, in the order the model asked.

Pass 1: sequential

For each tool request:

print the command, check for cut-off, **ask the guard**.

Why guard questions come first

A person can only answer one question at a time.

Questions one at a time, before anything runs.

The pending dict

Approved commands go into `pending` .

Denied or cut-off ones get their error result right away.

The parallel part

```
pool.map(execute, pending.values())
```

A `ThreadPoolExecutor` .

All approved commands run **at once**.

execute()

The Lesson 7 `docker exec` .

Plus the Lesson 5 trace.

One function. One command.

Pass 2: in order

Print each result.

Build the `tool_result` list in the **original order**.

Why order matters

Each `tool_result` names its `tool_use_id`.

They go back in the order the model asked: the API checks that.

Same results, no matter who finished first.

Run it

Start in a scratch folder: the box mounts it.

```
uv run --project /path/to/building-agents \
/path/to/building-agents/production/09-performance/quark.py \
"Run these three commands as three separate commands: 'sleep 3; echo lint ok', 'sleep 3; echo types ok', 'sleep 3; echo tests ok'. Then say what passed, in one line."
```

A note on running it

`sleep` isn't on the safe list.

The guard asks.

So the demos pipe `y` into stdin.

The comparison

Three tools that each sleep 3 seconds.

Lesson 8: **13.0 s** total.

Lesson 9: **6.9 s** total.

Where the time went

Lesson 9's tools ran together: about **3 seconds** each, overlapping.

The rest is the two model calls.

What stayed the same

The same answer.

The same commands.

Faster and cheaper.

Going further

`quark.py` is one way to do it.

`performance.py` is a richer one.

performance.py

Built on Lesson 3's `control_flow.py` .

Async: `AsyncAnthropic` .

The new idea: routing

Not every task needs the strongest model.

Pick the model **per task**.

Control flow, not model interface

This looks like a model-interface choice.

It isn't.

It's a decision about **the work**.

Remember Lesson 3

Routing: one call classifies, code sends the work down a path.

One of the workflow arrangements.

It's the same pattern

One cheap call classifies the task.

Code picks the path.

The path is a **model**.

Lesson 1's fallback

Lesson 1's fallback model only covered an **outage**.

Routing picks a model because of the **task**.

Different decision. Different primitive.

route()

Asks the small model: how hard is this task?

It answers with a **tier**.

Three tiers

- **quick:** Haiku
- **standard:** Sonnet, medium effort
- **deep:** Opus, high effort

TIERS

A dict from tier name to **model and effort**.

Easy to read. Easy to change.

Decided once per task

Not per step.

Why once

The cache is **per model**.

Switch models mid-task and you lose the cache.

Routing is a trade

Cheaper and faster on easy tasks.

But it can **guess wrong**.

It did, in the README

The parallel-tools task was routed to **deep**.

That's more than it needed.

The README says so, honestly.

Fixing a wrong router

Measure it.

That's Lesson 10.

A bigger cacheable prefix

`notes()` reads `README.md` and `docs/*.md`.

They go into the system prompt.

Why add notes

About **10.6k tokens**.

Past the cache minimum.

Now caching has something to work on.

Cold, then warm

First run: about half the input from cache.

Second run: **all of it.**

(Roughly 50%, then 100%.)

Streaming events

It watches the stream's events.

Records **time to first output**.

The number you feel.

Step lines

Each step prints the tokens by kind:

new, cached, written.

You see the cache working, live.

Parallel tools, async

`asyncio.gather` runs the bash calls together.

`Semaphore`, `AT_ONCE = 4`.

At most four at a time.

Why a limit

Fifty at once would exhaust the machine.

A cap keeps it safe.

The timing line

3 commands: 3.0s together, 9.0s one after another

Concurrent time vs the sum.

What performance can be

- **cache**: what's reused, how long
- **size**: what's trimmed or summarized
- **latency**: streamed or whole
- **model**: fixed, or routed per task
- **tools**: sequential or parallel

What to take away

Rule: performance is spread across the primitives: context caches and trims, the model interface streams, output runs tools in parallel, and control flow routes.

Notice what performance never does

It never gathers input.

It never changes the loop: same calls, same steps, same stops.

It changes how, not what.

And in quark.py

It never touches control flow.

The loop is the same.

Routing is the one place control flow shows up, in `performance.py`.

What's missing

The agent is faster and cheaper.

Is it still as **good**?

Nothing measures that.

Next: Evaluation

Lesson 10 measures whether it still works.